



## 本小节内容

### 2019 年 45 题

45. (16 分)已知  $f(n)=n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$ , 计算  $f(n)$  的 C 语言函数 fl 的源程序(阴影部分)及其在 32 位计算机 M 上的部分机器级代码如下:

```
int fl( int n ) {  
1 00401000 55      push    ebp  
  
    if ( n > 1 )  
11 00401018 83 7D 08 01    cmp     dword ptr [ebp+8], 1  
12 0040101C 7E 17          jle     fl+35h (00401035)  
    return n * fl(n-1);  
13 0040101E 8B 45 08        mov     eax, dword ptr [ebp+8]  
14 00401021 83 E8 01        sub     eax, 1  
15 00401024 50              push    eax  
16 00401025 E8 D6 FF FF FF  call    fl (00401000)  
19 00401030 0F AF C1        imul    eax, ecx  
20 00401033 EB 05          jmp     fl+3Ah (0040103a)  
    else return 1;  
21 00401035 B8 01 00 00 00  mov     eax, 1  
  
26 00401040 3B EC          cmp     ebp, esp  
  
30 0040104A C3              ret
```

其中, 机器级代码行包括行号、虚拟地址、机器指令和汇编指令, 计算机 M 按字节编址, int 型数据占 32

位。请回答下列问题:

- (1) 计算  $f(10)$  需要调用函数 fl 多少次? 执行哪条指令会递归调用 fl?
- (2) 上述代码中, 哪条指令是条件转移指令? 哪几条指令一定会使程序跳转执行?
- (3) 根据第 16 行 call 指令, 第 17 行指令的虚拟地址应是多少? 已知第 16 行 call 指令采用相对寻址方式, 该指令中的偏移量应是多少(给出计算过程)? 已知第 16 行 call 指令的后 4 字节为偏移量, M 采用大端还是小端方式?
- (4)  $f(13)=6\ 227\ 020\ 800$ , 但 fl(13) 的返回值为 1 932 053 504, 为什么两者不相等? 要使 fl(13) 能返回正确的结果, 应如何修改 fl 源程序?
- (5) 第 19 行 imul eax, ecx 表示有符号数乘法, 乘数为 R[ecx] 和 R[ecx], 当乘法器输出的高、低 32 位乘积之间满足什么条件时, 溢出标志 OF=1? 要使 CPU 在发生溢出时转异常处理, 编译器应在 imul 指令后加一条什么指令?

#### 答案解析:

- (1) 计算  $f(10)$  需要调用函数 fl 共 10 次执行, 第 16 行 call 指令会递归调用 fl。
- (2) 第 12 行 jle 指令是条件转移指令。第 16 行 call 指令、第 20 行 jmp 指令、第 30 行 ret 指令一定会使程序跳转执行。



(3)第 16 行 call 指令的下一条指令的地址为  $00401025H+5=0040102AH$ ，故第 17 行指令的虚拟地址是  $0040102AH$ 。call 指令采用相对寻址方式，即目标地址  $=(PC)+\text{偏移量}$ ，call 指令的目标地址为  $00401000H$ ，所以偏移量

$=\text{目标地址}-(PC)=00401000H-0040102AH=FFFFFFD6H$ 。根据第 16 行 call 指令的偏移量字段为  $D6FF FF FF$ ，可确定 M 采用小端方式。

(4) 因为  $f(13)=6\ 227\ 020\ 800$ ，大于 32 位 int 型数据可表示的最大值，因而  $f(13)$  的返回值是一个发生了溢出的结果。为使  $f(13)$  能返回正确结果，可将函数  $f1$  的返回值类型改为 double(或 long long 或 long double)。

(5) 若乘积的高 33 位为非全 0 或非全 1，则  $OF=1$  编译器应该在 imul 指令后加一条“溢出陷阱指令”，使得 CPU 自动查询溢出标志 OF，当  $OF=1$  时调出“溢出异常处理程序”。

乘法结果为 64 位，低 32 位的最高位是符号位，符号位会自动扩展到高 32 位，因此可以理解高 33 位都是符号位，如果高 33 位是全 0，说明是乘积结果正数，高 33 位都是 1，代表乘积结果是负数。

下面附带一下考研的 C 代码及对应汇编。

```
int f1(int n)
```

```
{
```

```
    if(n>1)
```

```
        return n*f1(n-1);
```

```
    else
```

```
        return 1;
```

```
}
```

```
int main() {
```

```
    f1(10);
```

```
    return 0;
```

```
}
```

下面是汇编代码，和考研的汇编有一点点差异（我的是 AMD 的 cpu，`mov %esp,%ebp`



是把 esp 放入 ebp)，但是原理全部是一致的。

```
int f1(int n)
{
    40150f: 90                nop

00401510 <_f1>:
    401510: 55                push    %ebp
    401511: 89 e5             mov     %esp,%ebp
    401513: 83 ec 18          sub     $0x18,%esp
    if(n>1)
    401516: 83 7d 08 01       cmpl    $0x1,0x8(%ebp)
    40151a: 7e 14             jle     401530 <_f1+0x20>
        return n*f1(n-1);
    40151c: 8b 45 08          mov     0x8(%ebp),%eax
    40151f: 83 e8 01          sub     $0x1,%eax
    401522: 89 04 24          mov     %eax,(%esp)
    401525: e8 e6 ff ff ff    call    401510 <_f1>
    40152a: 0f af 45 08       imul    0x8(%ebp),%eax
    40152e: eb 05             jmp     401535 <_f1+0x25>
    else
        return 1;
    401530: b8 01 00 00 00    mov     $0x1,%eax
}
    401535: c9                leave
    401536: c3                ret

00401537 <_main>:

int main() {
    401537: 55                push    %ebp
    401538: 89 e5             mov     %esp,%ebp
    40153a: 83 e4 f0          and     $0xfffffff0,%esp
    40153d: 83 ec 10          sub     $0x10,%esp
    401540: e8 ab 00 00 00    call    4015f0 <__main>
        f1(10);
    401545: c7 04 24 0a 00 00 00 movl    $0xa,(%esp)
    40154c: e8 bf ff ff ff    call    401510 <_f1>
        return 0;
    401551: b8 00 00 00 00    mov     $0x0,%eax
    401556: c9                leave
    401557: c3                ret
    401558: 66 90             xchg    %ax,%ax
```



40155a: 66 90

xchg %ax,%ax

40155c: 66 90

xchg %ax,%ax

40155e: 66 90

xchg %ax,%ax

## 【22】王道考研C语言课程内 容评价



为了更加了解大家的学习效果，不断提  
高课程质量，希望大家积极反馈哦

↓ 腾讯问卷